

Stoquify Agent Runtime External Inputs Operator Runbook

Execution artifact | 26 July 2026

Scope: Controlled Phase 2B preparation and later Phase 3 review

Authority granted by this document: None

Activation state: false / false / null

Phase 3 authority: false

Purpose

This runbook explains how real people and managed platforms provide the external inputs that Stoquify's repository cannot create truthfully. The external-input generator validates the handoff and tells operators which collector to run. The existing collectors remain the only permitted path into the operational release and credential registers.

Use:

```
npm run agent:external-inputs:report
npm run agent:external-inputs:gate
```

The report command is diagnostic. The gate command fails until every required external reference is complete. Neither command activates an agent.

1. Product and Security Approver Identities

Required people

- Product approver: accountable for pilot value, scope, cohort, expected outcomes, and stop conditions.
- Security approver: accountable for tenant isolation, RBAC, tool boundaries, managed secrets, prompt-injection controls, evidence handling, and incident readiness.

These must be two distinct, real identities from the authoritative company directory. Local test users, shared accounts, service accounts, placeholders, and one person acting in both roles are rejected.

Procedure

1. Freeze the reviewed release identity: package, version, commit, artifact digest, manifest hash, evidence-bundle hash, environment, tenant allowlist, and role allowlist.
2. Create separate Product and Security approval tasks in the approved governance system.
3. Give both approvers the same immutable release packet.
4. Each approver records APPROVED or REJECTED, their directory identity, decision timestamp, expiry, conditions, and a value-free approval reference.
5. Publish the signed governance evidence through the query-free HTTPS governance endpoint.
6. Configure the endpoint and secret through the deployment platform, then run:

```
npm run agent:governance:evidence:apply
npm run agent:operational-release:report
```

Accepted values

- directory:// or identity:// identity references
- approval:// decision references
- timestamps and release-binding hashes

Do not place names, email addresses, OAuth tokens, signatures, or approval document contents in the operational register.

Invalidation

Approval is recaptured when it expires or when the commit, artifact, manifest, evidence bundle, allowlist, role scope, or environment changes.

2. Six Primary and Backup Owner Assignments

Required responsibilities

1. ROLLOUT
2. ROLLBACK
3. SUPPORT
4. PILOT
5. SECURITY_INCIDENT
6. ON_CALL_BACKUP

Each responsibility needs a real primary and a different real backup. Twelve assignment slots must therefore be accepted even if company policy permits one person to hold primary responsibility for more than one role.

Procedure

1. The release manager selects primary and backup candidates from the authoritative directory.
2. Confirm that each person has the access, competence, and availability required by the role.
3. Give each person the current rollout, rollback, support, pilot, or incident runbook.
4. Record explicit acceptance, runbook version, acceptance timestamp, active coverage window, and escalation reference.
5. Publish the owner records through the governance evidence endpoint.
6. Run `npm run agent:governance:evidence:apply`.

Gate behavior

The gate rejects missing roles, duplicate role records, primary/backup self-substitution, synthetic identities, expired coverage, missing acceptance, and missing escalation references.

3. Evidence HTTPS Endpoints

Stoquify uses six channels because they prove different facts and have different owners.

Channel	Purpose	URL variable	Managed secret variable	Apply command
CI	Clean build, immutable artifact, browser report, deployment binding	STOQUIFY_AGENT_CI_RELEASE_EVIDENCE_URL	STOQUIFY_AGENT_CI_RELEASE_EVIDENCE_SECRET	npm run agent:ci-release:evidence:apply
Governance	Approvals, owner acceptance, directory authority	STOQUIFY_AGENT_GOVERNANCE_EVIDENCE_URL	STOQUIFY_AGENT_GOVERNANCE_EVIDENCE_SECRET	npm run agent:governance:evidence:apply
Scheduler	Managed schedule, workload identity, concurrency, deployment	STOQUIFY_AGENT_SCHEDULER_EVIDENCE_URL	STOQUIFY_AGENT_SCHEDULER_EVIDENCE_SECRET	npm run agent:scheduler:evidence:apply
Reconciler	Readiness, heartbeat, three completed windows	STOQUIFY_AGENT_RECONCILER_BASE_URL	STOQUIFY_AGENT_RECONCILER_SECRET	npm run agent:reconciler:evidence:apply
Alert	Delivery, acknowledgement, retry, dead letter, recovery, escalation	STOQUIFY_AGENT_ALERT_EVIDENCE_URL	STOQUIFY_AGENT_ALERT_EVIDENCE_SECRET	npm run agent:alert:evidence:apply
Credential	Rotation, restart, verification, revocation, rejection	STOQUIFY_AGENT_CREDENTIAL_EVIDENCE_URL	STOQUIFY_AGENT_CREDENTIAL_EVIDENCE_SECRET	npm run agent:credential-rotation:evidence:apply

Endpoint requirements

- HTTPS outside local development
- no URL username, password, query string, fragment, or signed URL
- authentication secret of sufficient strength, injected at runtime
- a request with an invalid credential returns 401
- an authorized request returns a bounded, sanitized evidence response
- response is release-bound, recent, independently attested, and free of secret values
- endpoint access and collector invocation are audited

Deployment sequence

1. Deploy each authority adapter in its owning platform.
2. Create one managed secret per channel; do not reuse one secret across all collectors.
3. Grant read-only evidence access to the collector identity.
4. Inject URL and secret into the one-time evidence-capture job.
5. Exercise invalid-auth and authenticated requests.

6. Run the narrow `:evidence:gate` command.
7. Run the corresponding `:evidence:apply` command.
8. Remove the job's secret access when the capture window closes if the platform supports just-in-time access.

4. Managed Workload and Collector Credentials

Required identities

- Reconciler workload identity
- Evidence-collector workload identity
- Secret-manager namespace
- Credential-rotation authority
- Credential inventory authority

The external-input manifest stores references only. Secret values stay in the managed secret service.

Rotation procedure

1. Inventory all 15 credential classes in the credential register.
2. Classify each as `ROTATED_AND_REVOKED`, `TEST_ONLY_CONFIRMED`, or `NOT_PRESENT_CONFIRMED`.
3. For a present credential, create a new version in the secret manager.
4. Update the dependent workload reference and restart or roll the workload.
5. Verify successful use of the new version.
6. Revoke the old version.
7. Verify that the old version is rejected.
8. Record value-free references and ordered timestamps.
9. Obtain Security approval and publish the credential authority response.
10. Run:

```
npm run agent:credential-rotation:evidence:apply
npm run agent:credential-rotation:gate
```

Never place the credential itself, an authorization header, connection string, database URL, private key, signed URL, or environment dump in evidence.

5. Deployment Target and Immutable Artifact

Required release identity

- package ID and release version
- exact 40-character commit SHA
- immutable artifact digest and reference
- deployment reference

- manifest, evidence-bundle, and browser-report hashes
- pilot tenant and role allowlist references

Procedure

1. Select the reviewer-approved commit from a clean checkout.
2. Run protected CI and all required release tests.
3. Build the inactive artifact once.
4. Publish it to an immutable registry and record the registry digest.
5. Deploy that exact digest to the internal-pilot target with activation disabled.
6. Run enabled-pilot Playwright certification against the deployed artifact without exposing it to non-allowlisted users.
7. Publish the CI and deployment attestation.
8. Run `npm run agent:ci-release:evidence:apply`.

Rebuilding from the same commit creates a different release artifact unless reproducibility is independently proven. Use the recorded digest, not a mutable tag such as `latest`.

6. Qualified Statutory Reviewer Approval

Required person

A qualified OHADA and country-pack reviewer who is independent of the code author and authorized by Stoquify's legal/compliance policy.

Procedure

1. Obtain the authoritative statutory source artifact.
2. Recompute its SHA-256 digest in the controlled review environment.
3. Record the reviewer's directory identity and current qualification reference.
4. Review the country-pack interpretation, effective dates, formulas, thresholds, declarations, fixtures, and stated non-claims.
5. Produce a signed approval artifact and compute its SHA-256 digest.
6. Update the statutory evidence manifest using the existing statutory workflow.
7. Run:

```
npm run statutory:country-pack:gate
```

The generator validates references and hashes; it cannot make a legal interpretation or substitute for qualified approval.

7. Phase 2B Pilot Cohort and Activation Authority

Required pilot definition

- named pilot program in the governance system

- value-free cohort reference
- hashed tenant allowlist
- hashed role allowlist
- support coverage
- rollback plan
- real activation authority
- signed activation decision reference

Procedure

1. Product proposes a minimal cohort and documents the value hypothesis and success metrics.
2. Security approves tenant and role boundaries.
3. Support and incident owners confirm coverage.
4. Engineering verifies that the deployed package remains inactive.
5. Enterprise gate 017 reviews the complete pre-activation evidence bundle.
6. After APPROVED_GO, the separate activation authority records a release-bound activation decision.
7. Conduct the protected activation ceremony for the approved cohort only.
8. Observe the bounded pilot, capture monitoring and support evidence, and exercise rollback.
9. Record run counts, incidents, and zero-tolerance safety counters.
10. Obtain distinct Product, Security, Finance-domain, and Release pilot-exit approvals.
11. Run `npm run agent:phase3:entry:gate`.

The external-input readiness gate can validate that activation authority exists, but it does not execute activation. The operational register remains false / false / null until the separate ceremony.

Authoritative Application Order

1. `agent:external-inputs:report`
2. `agent:ci-release:evidence:apply`
3. `agent:governance:evidence:apply`
4. `agent:scheduler:evidence:apply`
5. `agent:reconciler:evidence:apply`
6. `agent:alert:evidence:apply`
7. `agent:credential-rotation:evidence:apply`
8. `agent:credential-rotation:gate`
9. `agent:operational-release:gate`
10. `statutory:country-pack:gate`
11. `agent:phase2b:entry:gate`
12. independent enterprise gate 017 review
13. separate controlled activation ceremony
14. pilot exit capture and `agent:phase3:entry:gate`

Subsequent Interventions

Every later intervention follows the same rule:

1. Perform the real action in the authoritative system.
2. Publish a sanitized, signed, release-bound evidence response.

3. Run invalid-auth and freshness validation.
4. Apply through the narrow collector.
5. Rerun the narrow gate and then the composed gate.
6. Recapture evidence after expiry, rotation, ownership change, release change, or environment change.
7. Retain superseded evidence in the authoritative audit system.

No later intervention may silently overwrite authority, broaden tenant or role scope, insert secret values, or convert evidence readiness into automatic activation.